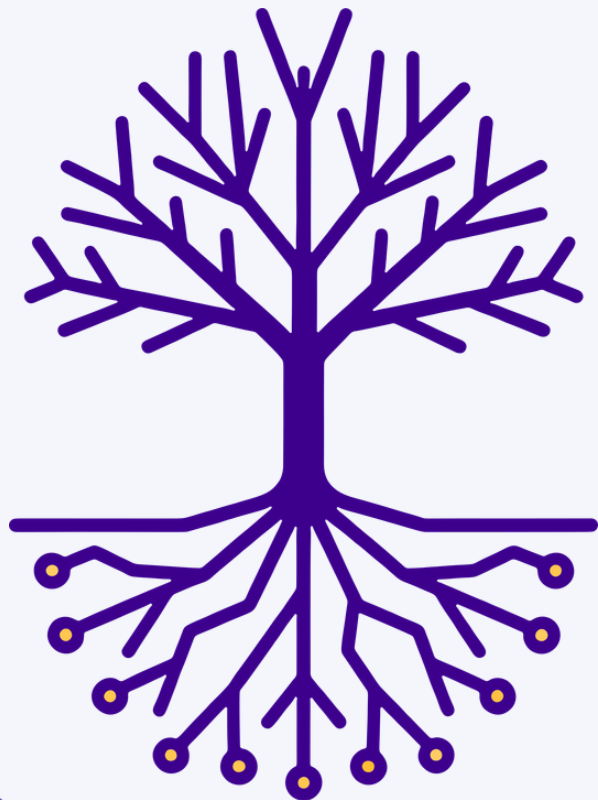


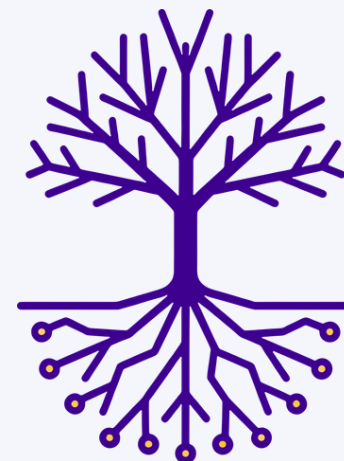


Árvores B+

ADS- MOD III
Sthefanny Brenda
Kauã Pereira
Adriel Barros
Eduardo Martins
Pedro Riquelme

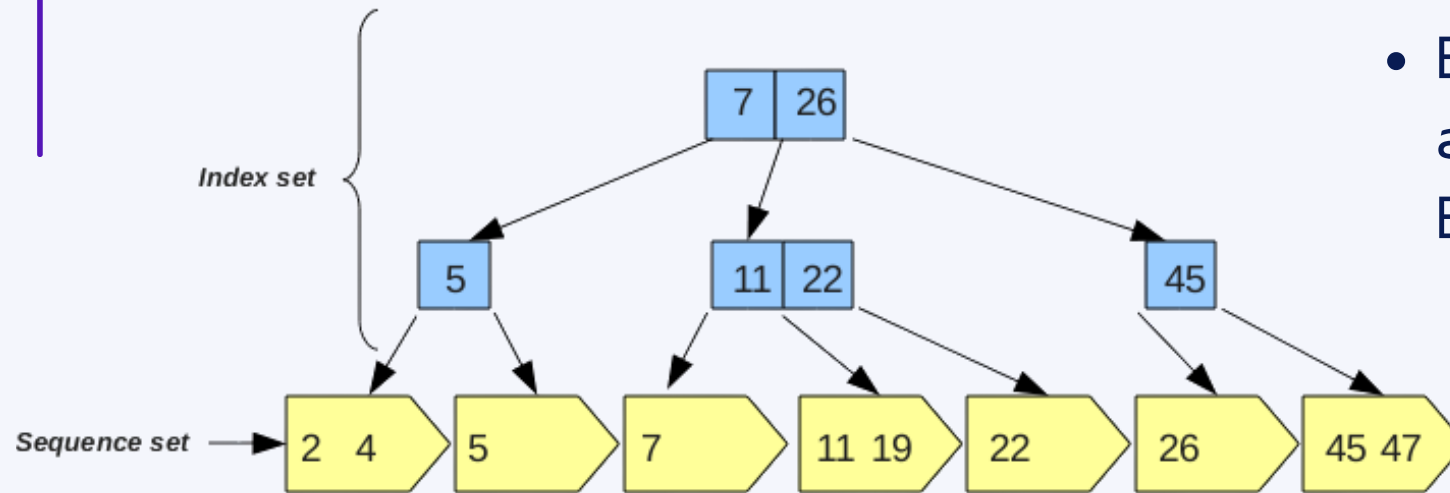
Tópicos:

- Introdução
- Estrutura do nó
- Operações
- Complexidade
- Vantagens e Desvantagens
- Comparação



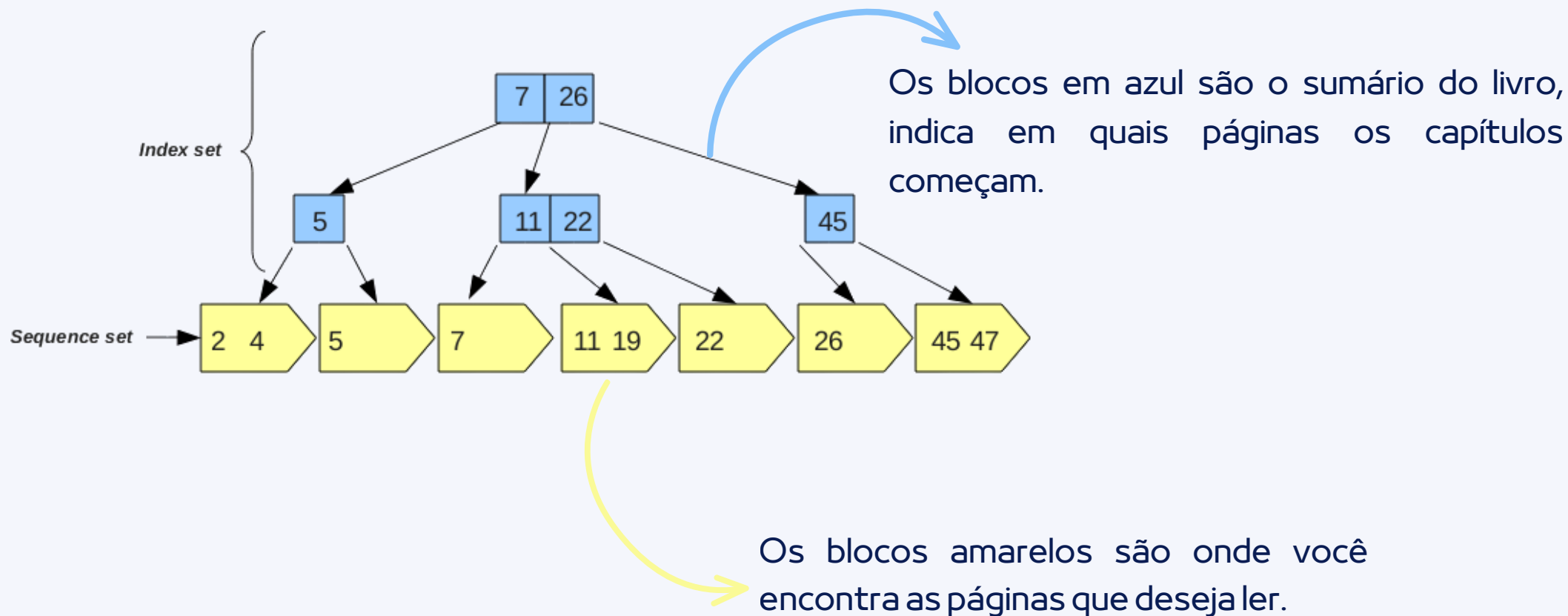
Introdução

O que é uma árvore B+?



- Estrutura de dados do tipo árvore derivada da Árvore tipo B
- Forma de armazenamento diferente (apenas nas folhas)
- Suas folhas são encadeadas

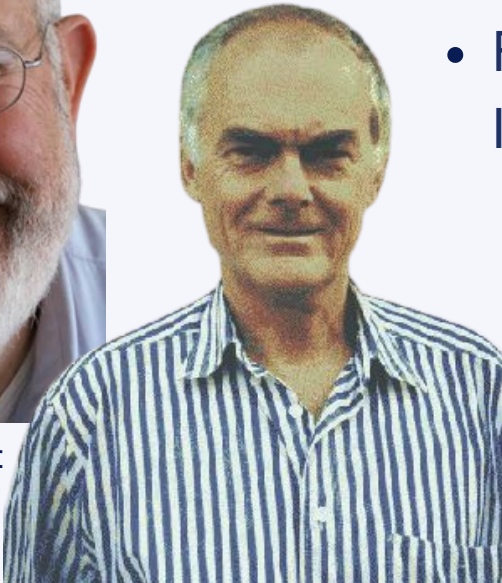
Pense nas Árvores B+ como um livro...



Motivação e Origem



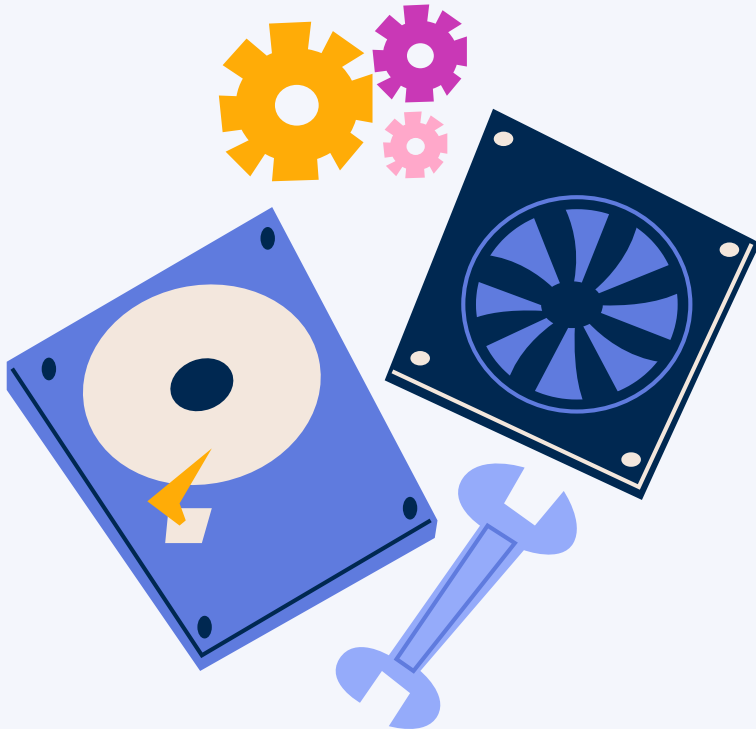
Edward M. McCreight



Rudolf Bayer

- A Árvore B+ deriva da Árvore B, criada por **Rudolf Bayer** e **Edward M. McCreight** por volta de 1972.
- Foi criada para melhorar o desempenho de sistemas que leem grandes blocos de dados.

O Problema que Resolve



- Problemas de eficiência de memória;
- Buscas lentas em discos;
- Demora para buscas sequenciais;
- Desperdício de memória.

Principais aplicações

Bancos de Dados

Índices primários e secundários em MySQL, PostgreSQL, Oracle e SQL Server.

Sistemas de Arquivos

NTFS, HFS+ e ext4 usam estruturas similares para localizar blocos de dados rapidamente.

Motores de Busca

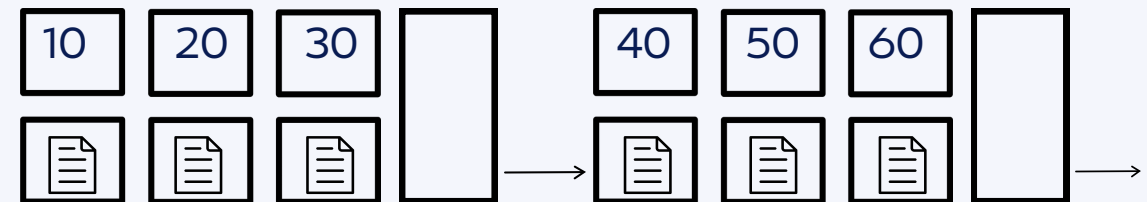
Indexação de grandes volumes de documentos para recuperação eficiente de registros.

Data Warehouses

Consultas analíticas sobre bilhões de registros com varreduras sequenciais otimizadas.



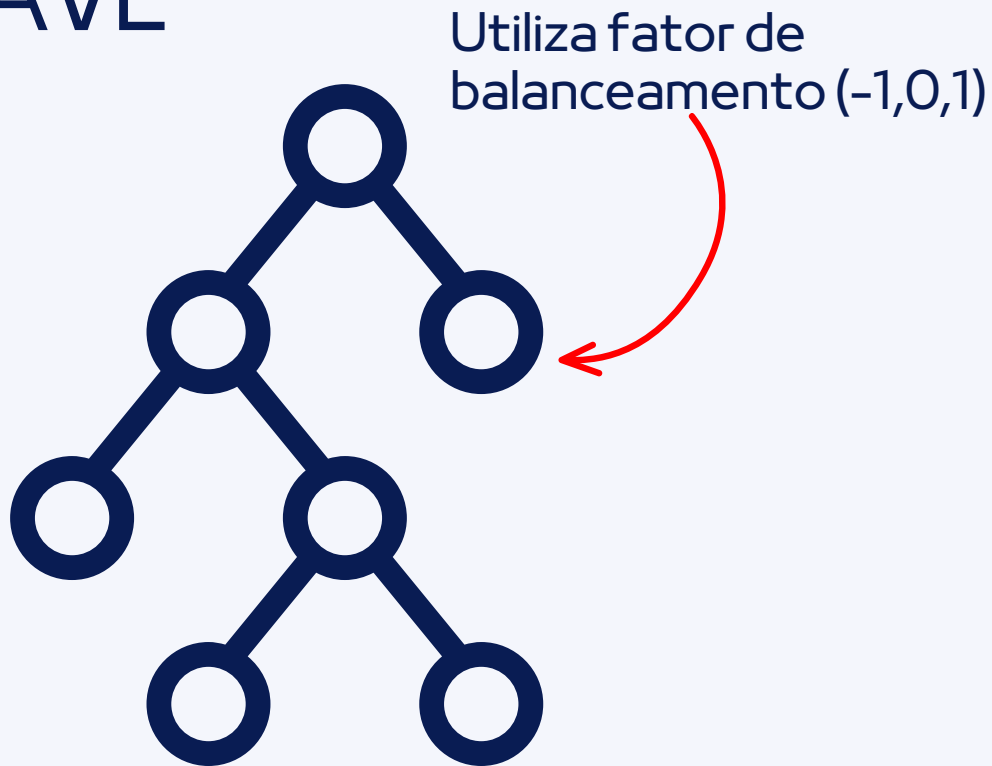
TITULO DO SLIDE



Ponteiro para a próxima folha
Liga uma folha a próxima, formando
uma lista encadeada de folhas

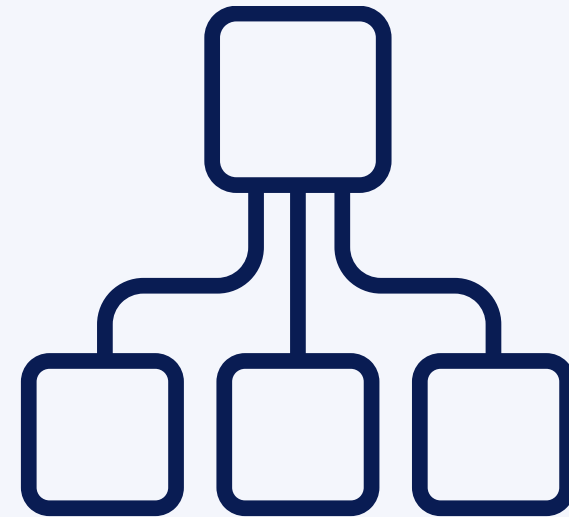
COMPARAÇÃO DE BALANCEAMENTO AVL VS ÁRVORE B+

AVL



O balanceamento é verificado a cada inserção ou remoção

ÁRVORE B+





TITULO DO SLIDE



TITULO DO SLIDE

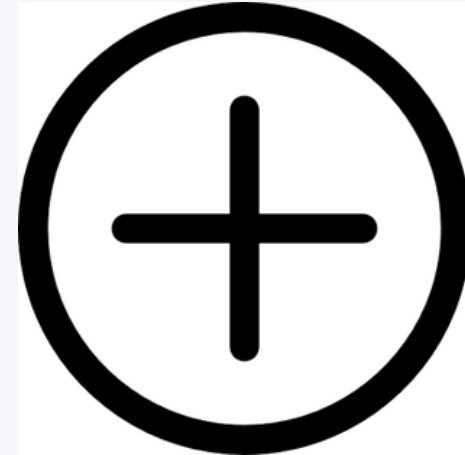


TITULO DO SLIDE

Inserção

Como ocorre:

- Localiza a folha correta
- Insere a nova chave
- Verifica overflow
- Realiza split, se necessário
- Atualiza o nó pai

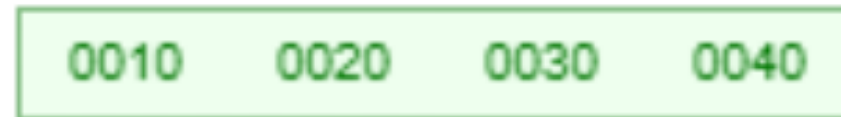


Pontos importantes:

- A inserção sempre ocorre nas folhas.
- As chaves permanecem em ordem.
- O split acontece apenas quando há overflow.
- A árvore permanece balanceada após a inserção.

Overflow

- Ocorre quando um nó cheio atinge sua capacidade máxima após a inserção de uma nova chave

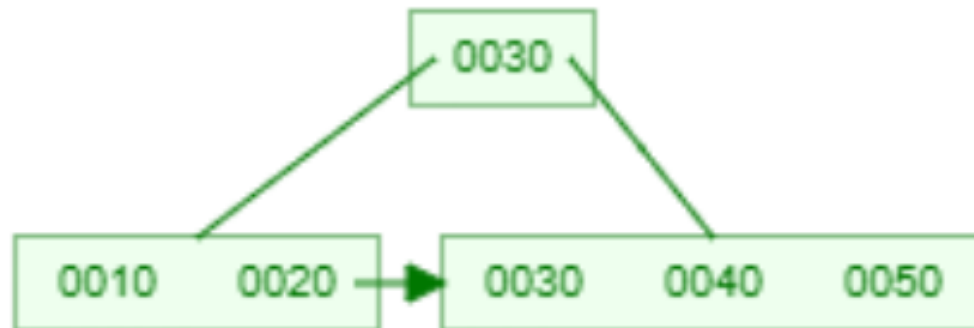


Split

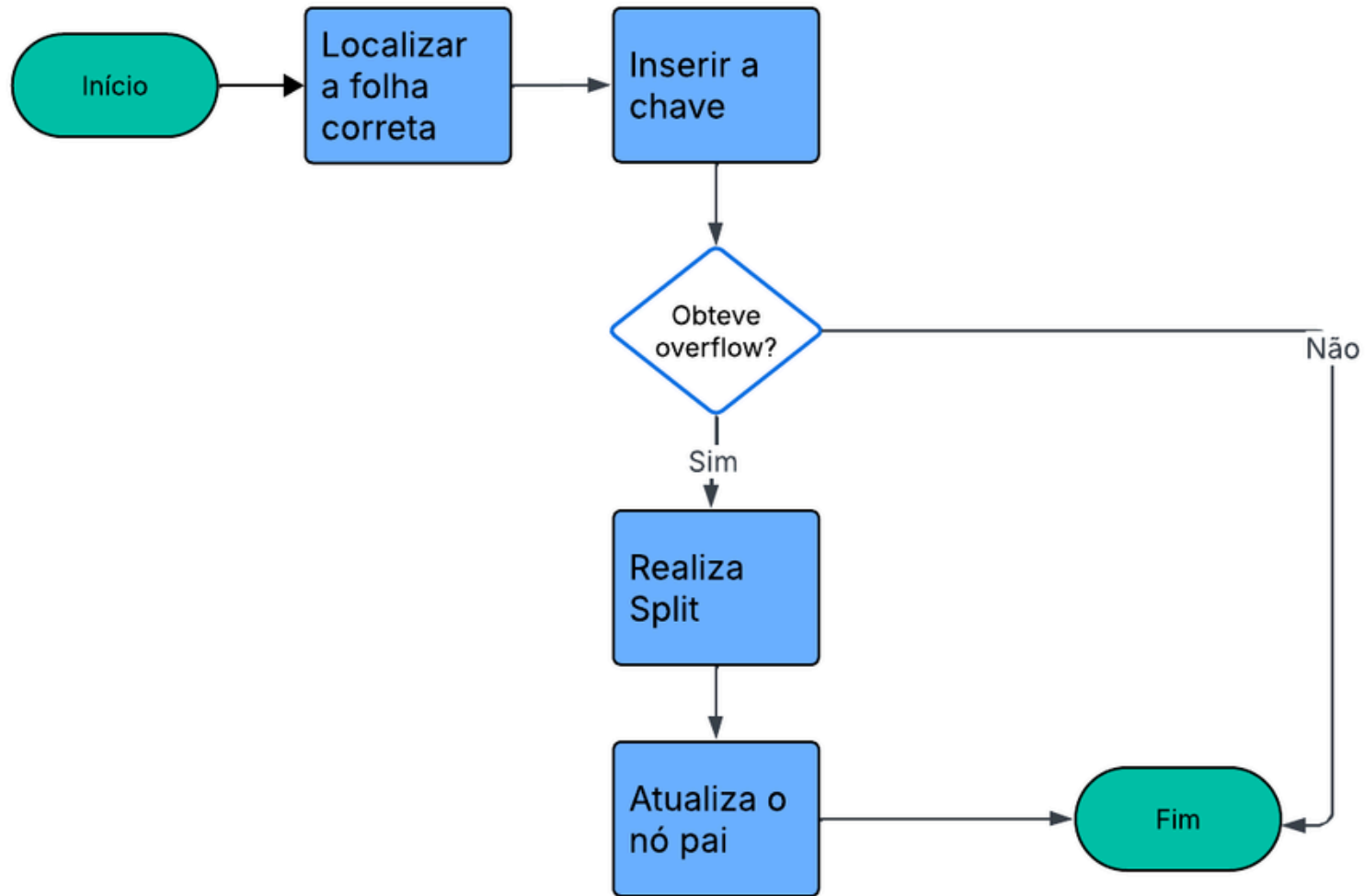
- Divide um nó cheio em dois e promover a chave central para o nó pai

0010 0020 0030 0040

0010 0020 0030 0040 0050

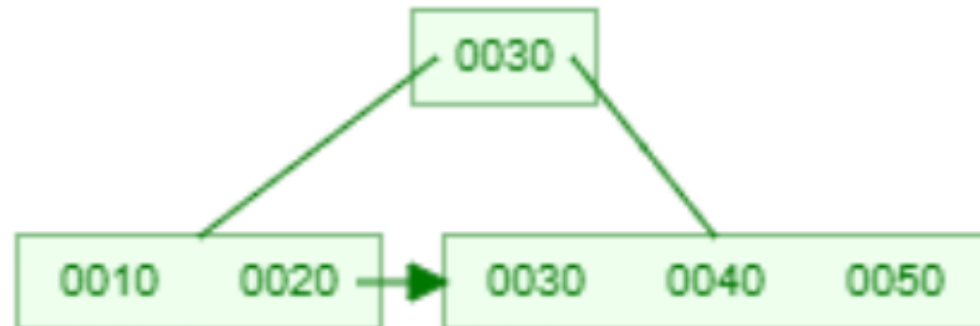
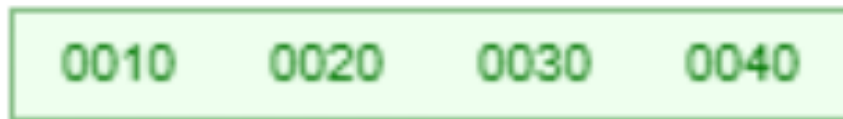


Inserção



Exemplo: Inserção

Inserção número 50 em grau maximo 5



Busca

Como ocorre:

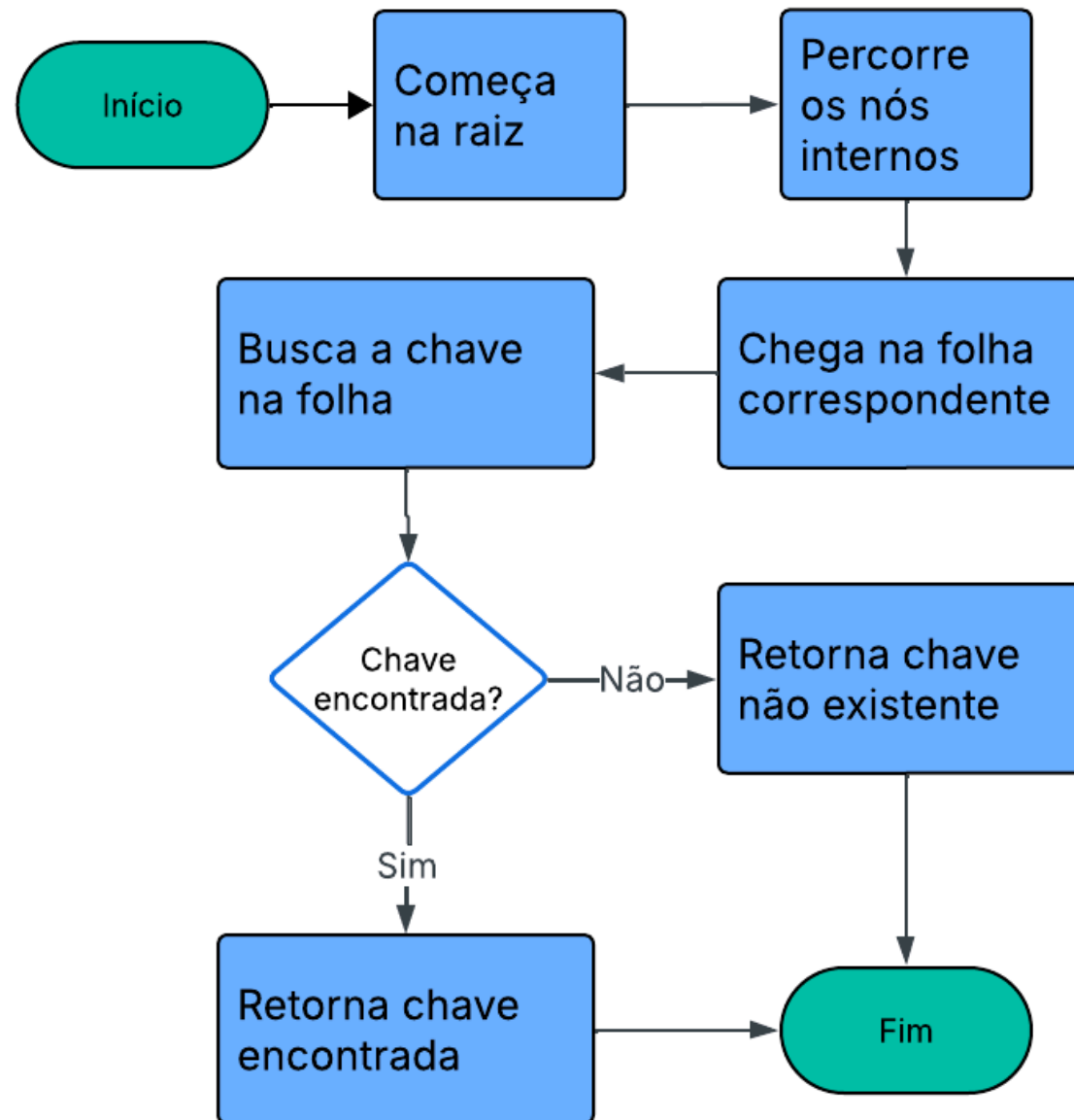
- Começa na raiz.
- Percorre os nós internos.
- Localiza a folha correspondente.
- Verifica se a chave está presente.
- Retorna o resultado da busca.



Pontos importantes:

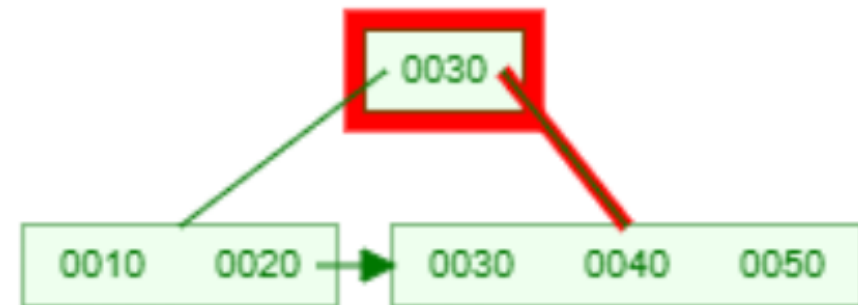
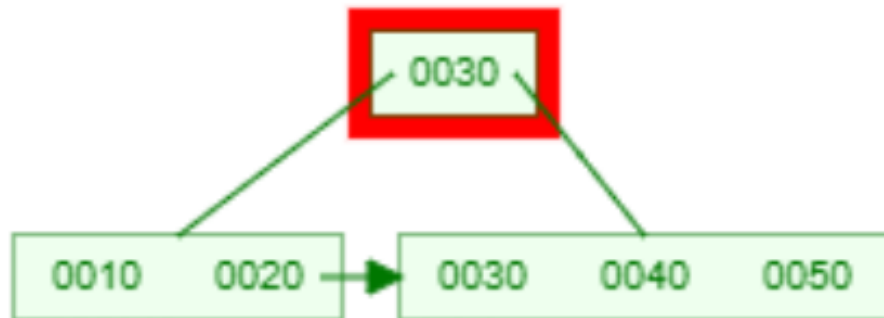
- A busca sempre inicia pela raiz.
- Os nós internos apenas direcionam o caminho.
- A chave é procurada apenas nas folhas.
- Se a chave não existir, a busca é encerrada.

Busca



Exemplo: busca

Busca número 40



Remoção

Como ocorre:

- Localiza a chave na folha.
- Remove o elemento.
- Verifica ocorrência de underflow.
- Realiza redistribuição ou merge, se necessário.
- Atualiza os nós internos.

Pontos importantes:

- A remoção sempre acontece nas folhas.



Underflow

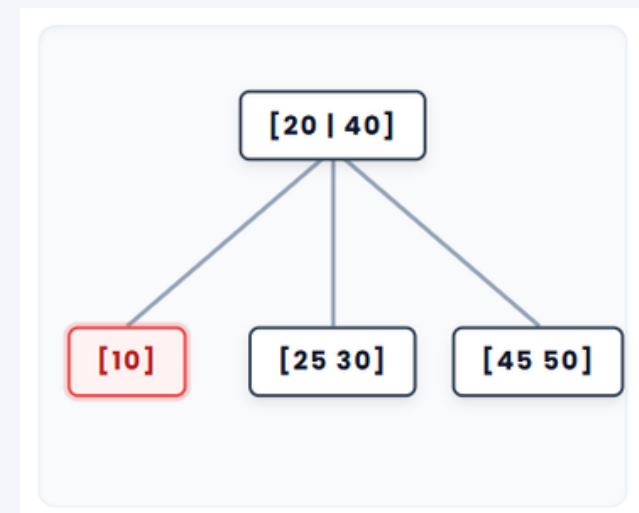
- Ocorre quando uma remoção reduz o número de elementos de um nó abaixo do limite mínimo permitido pela sua ordem.

Formúla:

Máximo de chaves = $m - 1$

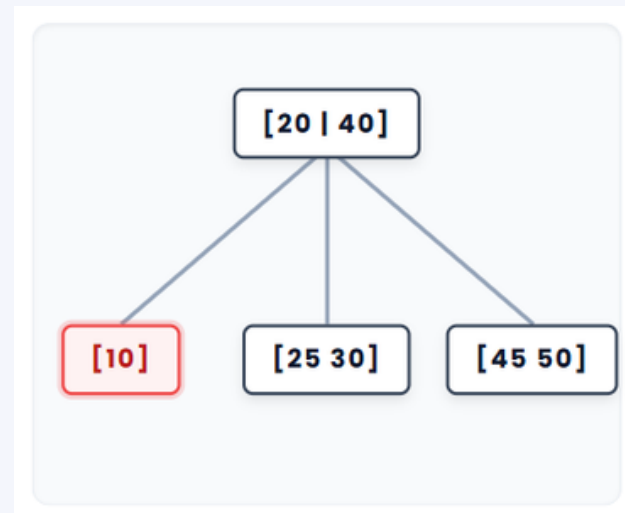
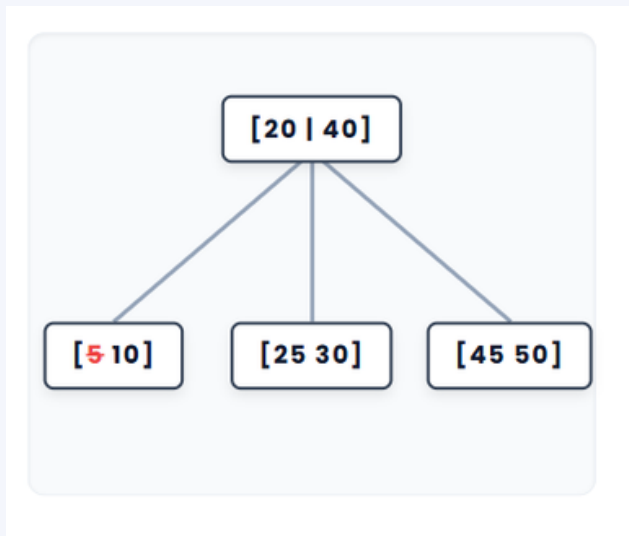
Mínimo de chaves = $\lceil m/2 \rceil - 1$

m = (quantidade máxima de filhos que um nó pode ter).



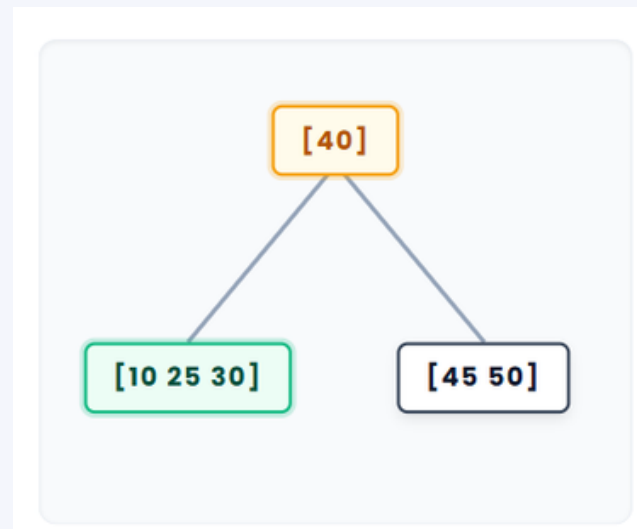
Exemplo: remoção

Remoção número 5



Redistribuição

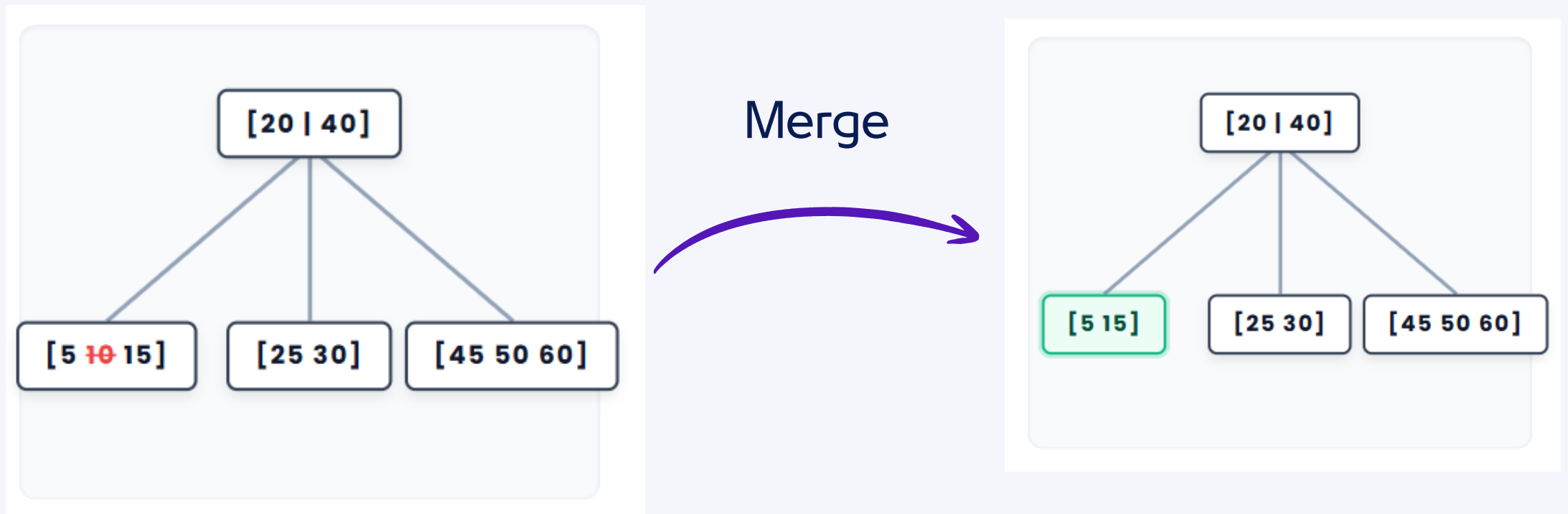
A redistribuição tenta
pegar uma chave de um
nó irmão.



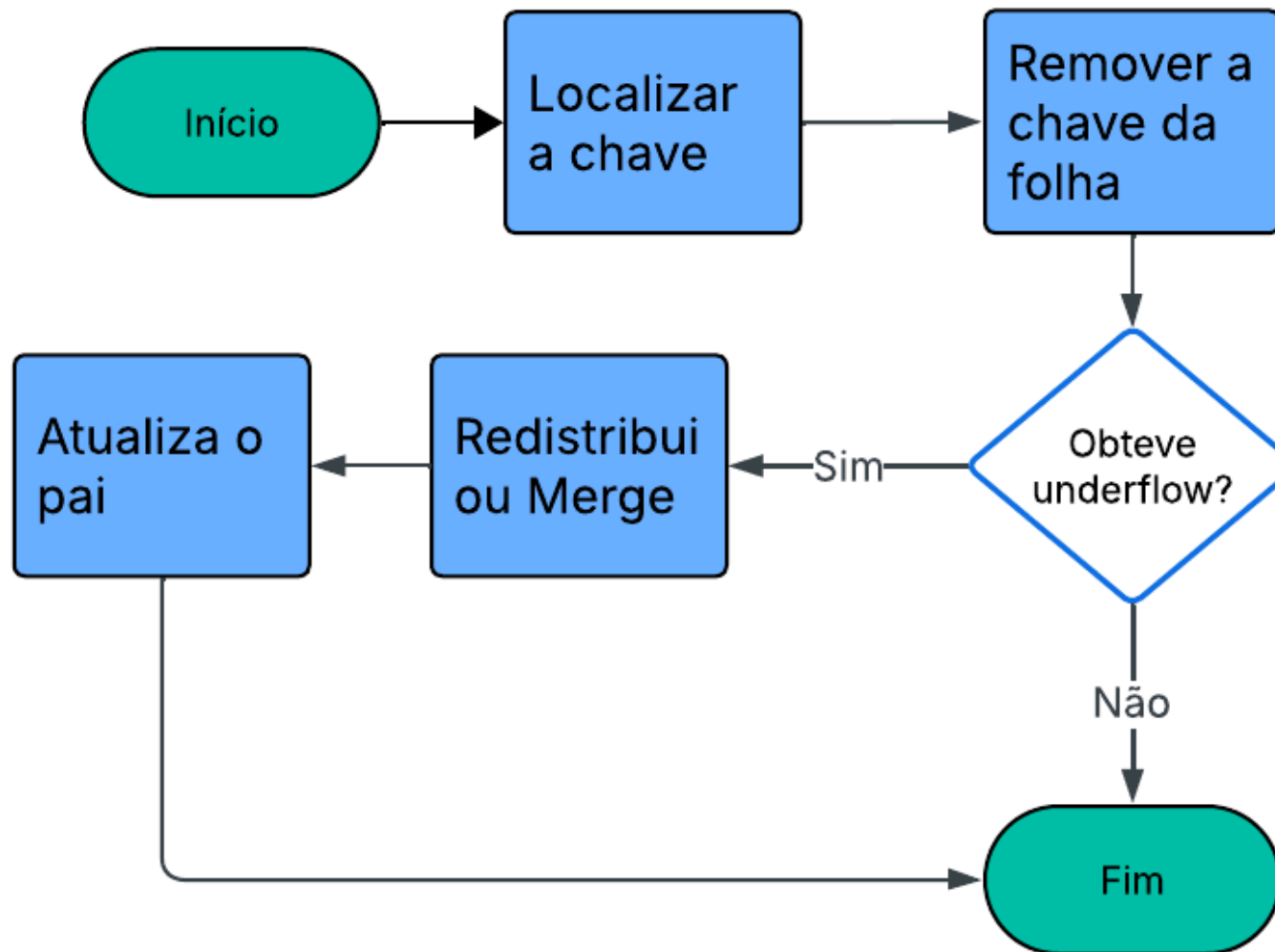
Exemplo: remoção

Remoção número 10

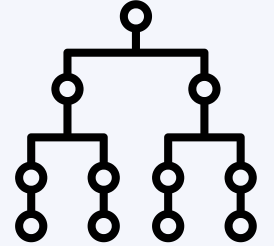
O merge une nós quando a redistribuição não é possível.



Remoção



Complexidade das Operações



As principais operações de uma árvore b+ possuem complexidade:

- Busca : $O(\log n)$
- Inserção: $O(\log n)$
- Remoção: $O(\log n)$

Motivo: todas dependem da altura da árvore, que cresce de forma logarítmica devido ao alto número de filhos por nó.

Por que a altura é $O(\log n)$?

Cada nó pode possuir vários filhos (m), fazendo com que a quantidade de nós aumente exponencialmente a cada nível:

- Nível 0 $\rightarrow 1$
- Nível 1 $\rightarrow m$
- Nível 2 $\rightarrow m^2$
- Nível 3 $\rightarrow m^3$

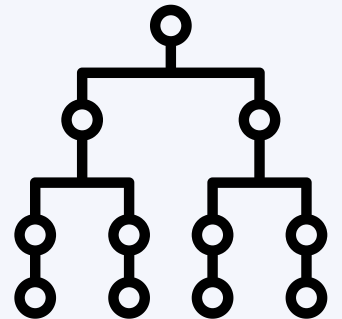
Conclusão: quanto maior o número de filhos por nó, menor a altura da árvore.

Busca: $O(\log n)$

- Percorre um caminho da raiz até uma folha.
- Visita no máximo (h) níveis da árvore

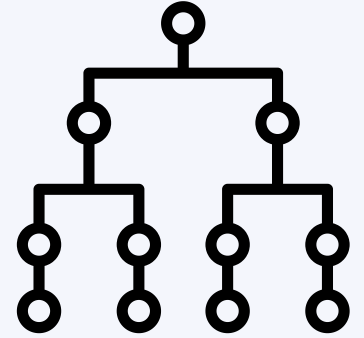
Como $h = O(\log n)$

$$\text{Busca} = O(\log n)$$



Inserção: $O(\log n)$

- Primeiro localiza a folha (**Busca**)
- Se necessario realiza os **split** dos nós

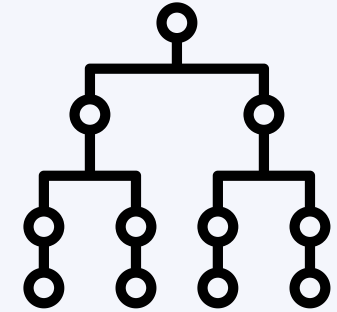


Mesmo no pior caso, o split ocorre apenas ao longo do caminho até a raiz

$$\text{Inserção} = O(\log n)$$

Remoção: $O(\log n)$

- Localiza a chave (**Busca**).
- Pode ocorrer **redistribuição** ou **merge**.

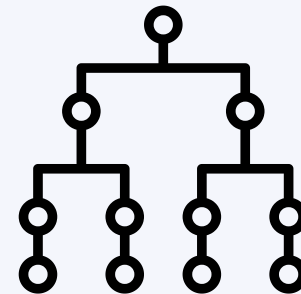


Redistribuição: A redistribuição ocorre quando um nó irmão possui chaves sobrando. Nesse caso, uma dessas chaves é transferida para o nó que ficou com poucas chaves, evitando a fusão dos nós.

Merge: O merge ocorre quando nenhum dos irmãos possui chaves para emprestar. Nesse caso, os dois nós são unidos em um único nó.

Remoção = $O(\log n)$

Resumo:



Busca	$O(\log n)$	Percorre apenas a altura da árvore
Inserção	$O(\log n)$	Busca + split em até um nó por nível.
Remoção	$O(\log n)$	Busca + merge/redistribuição em até um nó por nível.

Conclusão: Como a árvore B+ é balanceada e possui alto fator de ramificação, sua altura é $O(\log n)$. Por isso, todas as operações fundamentais apresentam complexidade logarítmica.

Remoção e Balanceamento

Remoção

Remove a chave da folha. Se o nó ficar com poucas chaves (abaixo de $\lceil m/2 \rceil - 1$), tenta redistribuir com um irmão ou faz merge (fusão) com ele.

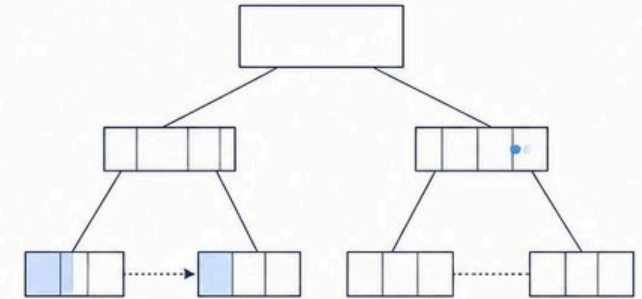
Balanceamento Automático

Split e merge garantem que todos os nós (exceto a raiz) tenham entre $m/2$ e m chaves.

A árvore permanece balanceada sem rebalanceamento explícito.

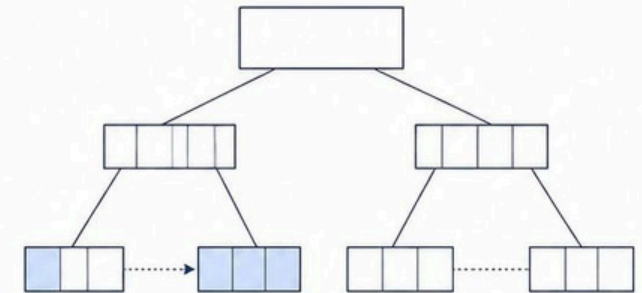
Três casos de remoção em Árvore B+:

Caso 1 - Redistribuição:
nó com déficit pega chave do irmão



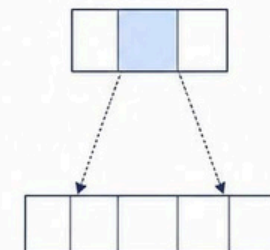
Três casos de remoção em Árvore B+:

Caso 2 - Merge:
nó subutilizado é fundido com irmão



Três casos de remoção em Árvore B+:

Caso 3 - Propagação:
merge na raiz reduz a altura da árvore



Análise de Desempenho

$$O(\log n)$$

Altura reduzida pelo alto fator de ramificação

$$O(\log n)$$

Split propaga no máximo até a raiz

$$O(\log n)$$

Merge/redistribuição percorre um caminho

$$O(n)$$

Folhas encadeadas permitem scan sequencial

O fator de ramificação m (tipicamente 100–500) reduz drasticamente a altura. Para 1 bilhão de chaves com $m=256$, a altura é apenas ~4 níveis — ou seja, no máximo 4 leituras de disco!

Quando Usar e Quando Evitar Uma Árvore B+?



É indicado usar a B+ quando

- Os dados são muito grandes e não cabem inteiramente na memória
- Precisa de índices eficientes em bancos de dados
- Varreduras sequenciais por intervalo são frequentes
- É necessário percorrer os dados em ordem, como listar todos os registros entre dois valores

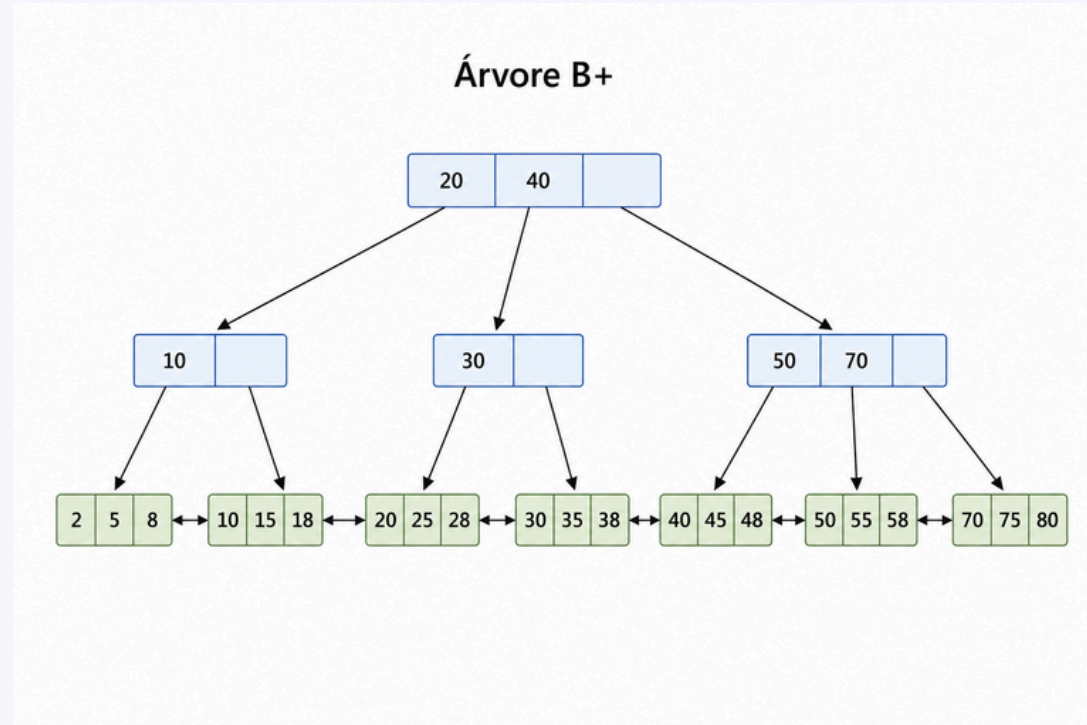


Evite usar ela quando

- Todos os dados cabem em memória RAM
- A estrutura é pequena (milhares de elementos)
- Quando não há necessidade de consultas por intervalo
- A implementação precisa ser simples e rápida

Quais problemas ela resolve?

- Evita o desbalanceamento da árvore
- Reduz o número de acessos ao disco
- Acelera buscas em grandes volumes de dados
- Reduz o tempo de manutenção dos índices



Diferença da B+ para as outras árvores

Árvores Binárias

- Cada nó possui no máximo 2 filhos
- Não existe regra para organizar os valores
- Estruturas hierárquicas

Árvores BST

- Cada nó possui apenas dois filhos
- Valores menores à esquerda e maiores à direita
- Busca em memória

Árvores AVL

- Cada nó possui apenas dois filhos
- Igual à BST, mas balanceada
- Busca rápida em memória

Árvores B+

- Cada nó pode possuir vários filhos
- Chaves nos nós internos e dados nas folhas
- Bancos de dados e sistemas de arquivos